

Web Technology

BCA — Semester 3 — 2024 — Answer Sheet

Subject Code: CACS205

Group B

Attempt any SIX questions. [6×5=30]

2. What is CSS? Explain the method of using CSS on the web page.

[2+3]

Answer: CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation and layout of HTML documents. It controls colors, fonts, spacing, positioning, and overall visual appearance, separating content from design. **Methods of Using CSS on a Web Page:** 1. **Inline CSS:** Styles are applied directly to individual HTML elements using the style attribute. `<p style="color: blue; font-size: 16px;">This is inline CSS.</p>` **Pros:** Quick for single elements. **Cons:** Hard to maintain, not reusable. 2. **Internal (Embedded) CSS:** Styles are defined within the `<style>` tag inside the `<head>` section of the HTML document. `<head> <style> p { color: blue; font-size: 16px; } h1 { background-color: yellow; } </style> </head>` **Pros:** Good for single-page styling. **Cons:** Not shared across multiple pages. 3. **External CSS:** Styles are written in a separate .css file and linked to HTML using the `<link>` tag. `<!-- HTML File --> <head> <link rel="stylesheet" type="text/css" href="styles.css"> </head> /* styles.css */ p { color: blue; font-size: 16px; } h1 { background-color: yellow; }` **Pros:** Reusable across multiple pages, easy to maintain, cached by browsers. **Cons:** Requires an extra HTTP request. **Cascading Priority:** Inline > Internal > External (when same selectors are used).

3. What is XML? Write rules to create XML document with example.

[1+4]

Answer: XML (eXtensible Markup Language) is a markup language designed to store and transport data in a self-describing, platform-independent format. Unlike HTML, which is used to display data, XML focuses on what data is. It is both human-readable and machine-readable. **Rules to Create an XML Document:**

1. **Prolog:** Optional XML declaration at the top: `<?xml version="1.0" encoding="UTF-8"?>`
2. **Root Element:** Every XML document must have exactly one root element that contains all other elements.
3. **Case Sensitivity:** Tags are case-sensitive. Opening and closing tags must match exactly.
4. **Proper Nesting:** All elements must be properly nested within each other.
5. **Closing Tags:** Every opening tag must have a corresponding closing tag. Empty elements can use self-closing syntax: `<element />`
6. **Attribute Values:** Attribute values must always be enclosed in quotes (single or double).
7. **Valid Names:** Element names must start with a letter or underscore; cannot contain spaces.
8. **No Reserved Characters:** Special characters like `<`, `>`, `&` must be escaped or placed in CDATA sections.

Example of a Well-Formed XML Document: `<?xml version="1.0" encoding="UTF-8"?> <library> <book id="b001"> <title>Web Technology</title> <author>John Smith</author> <price currency="USD">45.00</price> <year>2024</year> </book> <book id="b002"> <title>OOP in Java</title> <author>Jane Doe</author> <price currency="USD">50.00</price> <year>2023</year> </book> </library>`

4. Explain HTML form elements with suitable example.

[2+3]

Answer: HTML Form Elements are used to collect user input on a web page. A form is created using the `<form>` tag and can contain various input elements that allow users to enter text, make selections, and submit data to a server. **Common HTML Form Elements:**

Element	Description
<code><form></code>	Container for all form elements; defines action and method
<code><input></code>	Most versatile element — text, password, radio, checkbox, submit, etc.
<code><label></code>	Defines a label for an input element
<code><textarea></code>	Multi-line text input
<code><select></code>	Dropdown list
<code><option></code>	Items within a select dropdown
<code><button></code>	Clickable button
<code><fieldset></code>	Groups related elements
<code><legend></code>	Caption for a fieldset

Example — Complete HTML Form: `<!DOCTYPE html> <html> <head> <title>Registration Form</title> </head> <body> <h2>Student Registration</h2> <form action="register.php" method="POST"> <label for="fname">First Name:</label> <input type="text" id="fname" name="fname" required>

 <label for="pwd">Password:</label> <input type="password" id="pwd" name="pwd">

 <label>Gender:</label> <input type="radio" name="gender" value="male"> Male <input type="radio" name="gender" value="female"> Female

 <label>Hobbies:</label> <input type="checkbox" name="hobby" value="reading"> Reading <input type="checkbox" name="hobby" value="sports"> Sports

 <label for="country">Country:</label> <select id="country" name="country"> <option value="np">Nepal</option> <option value="in">India</option> <option value="us">USA</option> </select>

 <label for="address">Address:</label> <textarea id="address" name="address" rows="4" cols="30"></textarea>
 <input type="submit" value="Register"> <input type="reset" value="Clear"> </form> </body> </html>`

5. What is XPath? Write XSLT code to display your books information (Title, price, ISBN, subject code) store into XML.

[1+4]

Answer: XPath (XML Path Language) is a query language used to navigate and select nodes in an XML document. It uses path expressions to identify and locate specific parts of an XML document, similar to how file paths work in an operating system. XPath is used in XSLT, XQuery, and other XML technologies.

Common XPath Expressions:

- `/bookstore` — Selects the root element bookstore
- `//book` — Selects all book elements anywhere in the document
- `/bookstore/book[1]` — Selects the first book element
- `//title` — Selects all title elements
- `//book[@category='java']` — Selects books with category attribute equal to 'java'
- `//book[price>30]` — Selects books with price greater than 30

XML File (books.xml): `<?xml version="1.0" encoding="UTF-8"?> <?xml-stylesheet type="text/xsl" href="books.xsl"?> <bookstore> <book> <title>Web Technology</title> <price>500</price> <isbn>978-3-16-148410-0</isbn>`

`<subjectCode>CACS205</subjectCode> </book> <book> <title>OOP in Java</title> <price>650</price> <isbn>978-0-13-468599-1</isbn> <subjectCode>CACS204</subjectCode> </book> </bookstore>`

XSLT File (books.xsl): `<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"> <xsl:template match="/"> <html> <body> <h2>Book Information</h2> <table border="1"> <tr> <th>Title</th> <th>Price</th> <th>ISBN</th> <th>Subject Code</th> </tr> <xsl:for-each select="bookstore/book"> <tr> <td><xsl:value-of select="title"/></td> <td><xsl:value-of select="price"/></td> <td><xsl:value-of select="isbn"/></td> <td><xsl:value-of select="subjectCode"/></td> </tr> </xsl:for-each> </table> </body> </html> </xsl:template> </xsl:stylesheet>`

6. Why server side programming is important for web development? Explain with its major features.

[5]

Answer: Importance of Server-Side Programming in Web Development:

Server-side programming is essential because it enables dynamic content generation, data processing, and business logic execution on the server before sending responses to clients. Without server-side processing, websites would be limited to static HTML pages that cannot interact with databases, authenticate users, or personalize content. **Key Reasons for Importance:**

1. **Dynamic Content Generation:** Server-side scripts generate HTML dynamically based on user input, database queries, or business rules.
2. **Database Interaction:** Enables CRUD (Create, Read, Update, Delete) operations on databases to persist and retrieve data.
3. **Security:** Sensitive operations (authentication, payment processing, data validation) occur on the server, hiding implementation details from users.
4. **User Authentication and Authorization:** Manages login sessions, access control, and user privileges securely.
5. **Cross-Platform Compatibility:** Server logic runs independently of the client's browser or device.

Major Features of Server-Side Programming:

Feature	Description
Session Management	Tracks user state across multiple page requests using sessions and cookies
Database Connectivity	Connects to MySQL, PostgreSQL, MongoDB, etc. using drivers or ORMs
File Handling	Reads, writes, and manipulates files on the server filesystem
API Development	Creates RESTful or SOAP APIs for client-server communication
Security Features	Encryption, input sanitization, CSRF/XSS protection, SSL/TLS support
Template Engines	Separates presentation logic from business logic (e.g., JSP, Blade, EJS)
Error Handling	Centralized logging and error reporting for debugging and monitoring
Scalability	Supports load balancing, caching, and distributed architectures

Popular Server-Side Technologies: PHP, Node.js, Python (Django/Flask), Java (Spring/Servlets/JSP), ASP.NET, Ruby on Rails.

7. Why session is important for web development, write a server side script to create, store, retrieve and remove session and cookie.

[2+3]

Answer: Importance of Session in Web Development:

HTTP is a stateless protocol, meaning each request from a client to a server is independent and does not retain any memory of previous interactions. **Sessions** solve this problem by allowing the server to maintain stateful information about a user across multiple requests. **Why Sessions Are Important:**

1. **User Authentication:** Keeps users logged in as they navigate between pages.
2. **Shopping Carts:** E-commerce sites use sessions to store cart items temporarily.
3. **Personalization:** Remembers user preferences, language settings, and themes.
4. **Security:** Stores sensitive data server-side rather than exposing it in cookies.
5. **Tracking:** Tracks user activity and progress through multi-step processes.

PHP Server-Side Script for Session and Cookie Management:

```
<?php // ===== CREATE AND STORE SESSION =====
session_start(); // Store data in session $_SESSION['username'] = 'ram_sharma'; $_SESSION['role'] = 'student'; $_SESSION['login_time'] = date('Y-m-d H:i:s'); echo "<h3>Session Created Successfully</h3>";
echo "Username: " . $_SESSION['username'] . "<br/>"; echo "Role: " . $_SESSION['role'] . "<br/>"; echo "Login Time: " . $_SESSION['login_time'] . "<br/>"; // ===== CREATE AND STORE COOKIE =====
// setcookie(name, value, expire, path, domain, secure, httponly) setcookie("user_pref", "dark_mode", time() + (86400 * 30), "/"); setcookie("visit_count", "1", time() + (86400 * 30), "/"); echo "<h3>Cookies Set Successfully</h3>"; // ===== RETRIEVE SESSION DATA =====
echo "<h3>Retrieve Session Data</h3>"; if (isset($_SESSION['username'])) { echo "Session Username: " . $_SESSION['username'] . "<br/>"; } else { echo "No session data found.<br/>"; } // ===== RETRIEVE COOKIE DATA =====
echo "<h3>Retrieve Cookie Data</h3>"; if (isset($_COOKIE['user_pref'])) { echo "Cookie Preference: " . $_COOKIE['user_pref'] . "<br/>"; } else { echo "No cookie found.<br/>"; } // ===== REMOVE SESSION DATA =====
echo "<h3>Remove Session</h3>"; // Remove specific session variables unset($_SESSION['username']); unset($_SESSION['role']); // Destroy entire session session_destroy(); echo "Session destroyed successfully.<br/>"; // ===== REMOVE COOKIE =====
echo "<h3>Remove Cookie</h3>"; // Set expiration time to past to delete cookie setcookie("user_pref", "", time() - 3600, "/"); setcookie("visit_count", "", time() - 3600, "/"); echo "Cookies removed successfully.<br/>"; ?>
```

Key Differences:

- **Session:** Data stored on the server; session ID sent to client via cookie. More secure for sensitive data.
- **Cookie:** Data stored on the client's browser. Limited size (~4KB) and can be disabled by users.

8. Explain multi-tier technology with its advantage and disadvantages.

[2+3]

Answer: Multi-Tier Architecture is a client-server architecture in which the presentation, application processing, and data management functions are physically separated into distinct layers or tiers. The most common implementation is the **Three-Tier Architecture**, consisting of Presentation Tier, Application/Business Logic Tier, and Data Tier. **Types of Multi-Tier Architecture:**

1. **One-Tier:** All layers (UI, logic, data) exist on a single system (e.g., desktop application).
 2. **Two-Tier (Client-Server):** Client handles presentation and some logic; server manages the database.
 3. **Three-Tier:** Separate presentation, application logic, and data tiers.
 4. **N-Tier:** Further breakdown into multiple specialized layers (web server, app server, cache layer, message queue, etc.).
- Three-Tier Architecture: TierNameRoleExamples**
- | | | |
|----------------------|--------------------------------|------------------------------|
| 1stPresentation Tier | User interface and interaction | HTML, CSS, JavaScript, React |
| 2ndApplication Tier | Business logic and processing | PHP, Java, Python, Node.js |
| 3rdData Tier | Data storage and retrieval | MySQL, PostgreSQL, MongoDB |

Advantages of Multi-Tier Technology:

1. **Scalability:** Each tier can be scaled independently based on demand.
2. **Maintainability:** Changes in one tier do not affect others, making updates easier.
3. **Reusability:** Business logic can be reused by multiple presentation layers (web, mobile, desktop).
4. **Security:** The data tier is isolated from direct client access, reducing attack surface.
5. **Performance:** Caching and load balancing can be applied at specific tiers.
6. **Flexibility:** Different technologies can be used for each tier based on requirements.
7. **Fault Tolerance:** Failure in one tier does not necessarily bring down the entire system.

Disadvantages of Multi-Tier Technology:

1. **Complexity:** More complex to design, develop, and deploy than monolithic architectures.
2. **Performance Overhead:** Network communication between tiers can introduce latency.
3. **Cost:** Requires more hardware, software licenses, and infrastructure.
4. **Deployment Challenges:** Coordinating deployments across multiple servers is more difficult.
5. **Testing Difficulty:** Integration testing becomes more complex with distributed components.
6. **Network Dependency:** Heavily reliant on network reliability and bandwidth between tiers.

Group C

Attempt any TWO questions. [2×10=20]

9. Write a server side script for implementing login. Your program should take username and password as input and redirect to home page if validated otherwise prints an error try again.

[10]

Answer: PHP Server-Side Login Implementation: login.php — Login Form: `<!DOCTYPE html> <html>
<head> <title>Login Page</title> <style> body { font-family: Arial; background: #f0f0f0; } .login-box { width:
300px; margin: 100px auto; padding: 20px; background: white; border-radius: 8px; } input { width: 100%;
padding: 10px; margin: 5px 0; } button { width: 100%; padding: 10px; background: #007bff; color: white;
border: none; } .error { color: red; margin-top: 10px; } </style> </head> <body> <div class="login-box">
<h2>User Login</h2> <form method="POST" action="authenticate.php"> <input type="text"
name="username" placeholder="Username" required> <input type="password" name="password"
placeholder="Password" required> <button type="submit">Login</button> </form> <?php if
(isset($_GET['error'])) { echo '<p class="error">Invalid username or password. Please try again.</p>'; } ?>
</div> </body> </html>` **authenticate.php — Validation Script:** `<?php session_start(); // Predefined valid
credentials (in real apps, check against database) $valid_username = "admin"; $valid_password =
"admin123"; // In production, use hashed passwords // Retrieve form data $username = $_POST['username']
?? "; $password = $_POST['password'] ?? "; // Validate credentials if ($username === $valid_username &&
$password === $valid_password) { // Store user info in session $_SESSION['username'] = $username;
$_SESSION['logged_in'] = true; $_SESSION['login_time'] = date('Y-m-d H:i:s'); // Redirect to home page on
successful login header("Location: home.php"); exit(); } else { // Redirect back to login with error message
header("Location: login.php?error=1"); exit(); } ?>` **home.php — Home Page (Protected):** `<?php
session_start(); // Check if user is logged in if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !==
true) { header("Location: login.php"); exit(); } ?>` `<!DOCTYPE html> <html> <head><title>Home</title></head>
<body> <h1>Welcome, <?php echo htmlspecialchars($_SESSION['username']); ?>!</h1> <p>You
successfully logged in at <?php echo $_SESSION['login_time']; ?>.</p> Logout
</body> </html>` **logout.php — Logout Script:** `<?php session_start(); session_unset(); session_destroy();
header("Location: login.php"); exit(); ?>` **Explanation:**

- **login.php** displays the login form with username and password fields.
- **authenticate.php** validates credentials against predefined values (or database in production).
- On success, it creates a session and redirects to **home.php** using `header("Location: ...")`.
- On failure, it redirects back to `login.php` with an error flag.
- **home.php** verifies the session before displaying content, preventing unauthorized access.
- **logout.php** destroys the session and redirects to the login page.

10. Compare XML Schema with DTD. Write XML schema to describe student as root element, name as child element with first-name, middle name, last name, city as child elements with its content "KTM","PKR","LPT" and rollno as its attribute.

[3+7]

Answer: Comparison of XML Schema (XSD) and DTD: Feature

DTD (Document Type Definition)	XML Schema (XSD)
Syntax	Uses non-XML syntax
Uses XML syntax	(valid XML document)
Data Types	Limited to text strings only (CDATA, PCDATA)
Rich set of data types	(string, int, date, decimal, boolean, etc.)
Namespace Support	No namespace support
Full namespace support	Complex Types
Limited support for complex structures	Supports complex types, sequences, choices, and groups
Occurrence Constraints	Only *, +, ? operators
minOccurs and maxOccurs for precise control	Extensibility
Not extensible	Highly extensible with inheritance and derivation
Validation	Less powerful validation
Strong and detailed validation capabilities	Learning Curve
Simpler to learn	More complex but more powerful
Standard	Older SGML-based standard
W3C Recommendation, modern standard	

XML Schema for Student:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- Define city enumeration -->
  <xs:simpleType name="cityType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="KTM"/>
      <xs:enumeration value="PKR"/>
      <xs:enumeration value="LPT"/>
    </xs:restriction>
  </xs:simpleType>
  <!-- Define name complex type -->
  <xs:complexType name="nameType">
    <xs:sequence>
      <xs:element name="first-name" type="xs:string"/>
      <xs:element name="middle-name" type="xs:string"/>
      <xs:element name="last-name" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
  <!-- Define student element -->
  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="nameType"/>
        <xs:element name="city" type="cityType"/>
      </xs:sequence>
      <xs:attribute name="rollno" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Valid XML Instance Document:

```
<?xml version="1.0" encoding="UTF-8"?>
<student xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="student.xsd"
rollno="BCA101">
  <name>
    <first-name>Ram</first-name>
    <middle-name>Prasad</middle-name>
    <last-name>Sharma</last-name>
  </name>
  <city>KTM</city>
</student>
```

Explanation:

- cityType restricts city values to "KTM", "PKR", or "LPT" using enumeration.
- nameType is a complexType containing first-name, middle-name, and last-name as a sequence.
- The student element contains a name element and a city element, plus a required rollno attribute.
- use="required" ensures rollno must be present in every student element.

11. Create web page with following details. a) Create basic structure [2] b) Add header section to the top of page which contain logo and search bar. [2] c) Create two column layout of (25%,75%) width and put unordered list into first one and image gallery into second one. [2] d) Add footer section, which includes address of company and copyright information. [2] e) Also add some basic CSS using external stylesheet.[2]

[2+2+2+2+2]

Answer: index.html: `<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>My Company</title> <link rel="stylesheet" href="style.css"> </head> <body> <!-- Header Section --> <header> <div class="logo">MyCompany</div> <div class="search-bar"> <input type="text" placeholder="Search..."> <button>Search</button> </div> </header> <!-- Main Content: Two Column Layout --> <div class="main-container"> <aside class="sidebar"> <h3>Navigation</h3> Home About Us Services Portfolio Contact </aside> <section class="content"> <h2>Image Gallery</h2> <div class="gallery"> </div> </section> </div> <!-- Footer Section --> <footer> <div class="address"> <p>MyCompany Pvt. Ltd.</p> <p>123 Business Street, Kathmandu, Nepal</p> <p>Phone: +977-1-1234567 | Email: info@mycompany.com</p> </div> <div class="copyright"> <p>© 2024 MyCompany. All rights reserved.</p> </div> </footer> </body> </html>`

style.css (External Stylesheet): `/* Reset and Base Styles */ * { margin: 0; padding: 0; box-sizing: border-box; } body { font-family: Arial, sans-serif; line-height: 1.6; background-color: #f4f4f4; } /* Header Styles */ header { background-color: #333; color: white; padding: 15px 20px; display: flex; justify-content: space-between; align-items: center; } .logo { font-size: 24px; font-weight: bold; } .search-bar input { padding: 8px; width: 200px; border: none; border-radius: 4px 0 0 4px; } .search-bar button { padding: 8px 15px; background-color: #007bff; color: white; border: none; border-radius: 0 4px 4px 0; cursor: pointer; } /* Main Container: Two Column Layout */ .main-container { display: flex; min-height: 400px; margin: 20px; } .sidebar { width: 25%; background-color: #fff; padding: 20px; border-radius: 5px; box-shadow: 0 0 5px rgba(0,0,0,0.1); } .sidebar h3 { margin-bottom: 10px; color: #333; } .sidebar ul { list-style-type: disc; padding-left: 20px; } .sidebar ul li { margin: 8px 0; } .sidebar ul li a { text-decoration: none; color: #007bff; } .sidebar ul li a:hover { text-decoration: underline; } .content { width: 75%; padding: 20px; background-color: #fff; margin-left: 20px; border-radius: 5px; box-shadow: 0 0 5px rgba(0,0,0,0.1); } .content h2 { margin-bottom: 15px; color: #333; } .gallery { display: grid; grid-template-columns: repeat(2, 1fr); gap: 15px; } .gallery img { width: 100%; height: 150px; object-fit: cover; border-radius: 5px; border: 1px solid #ddd; } /* Footer Styles */ footer { background-color: #333; color: white; text-align: center; padding: 20px; margin-top: 20px; } .address p { margin: 5px 0; } .copyright { margin-top: 10px; font-size: 14px; color: #ccc; }`

Explanation:

- **Basic Structure (a):** Proper DOCTYPE, html, head, and body tags with meta charset and viewport for responsiveness.
- **Header (b):** Contains a logo div and a search bar with input field and button.
- **Two-Column Layout (c):** Uses CSS Flexbox. Sidebar is 25% with unordered list; Content is 75% with image gallery using CSS Grid.
- **Footer (d):** Contains company address and copyright information.
- **External CSS (e):** All styling is in style.css, linked via <link> tag in the head section.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.